

MODULE 24 · PART 8 - PRACTICE

Twelve Unsolved

The same exercise, with the answers taken out.

PART 8 of 8 MODULE 24 of 26 RUNS about 22 min

An answer that arrives five seconds later is **recognition**. You want **retrieval**.

How to work through these

- ◆ **Pause on each one and commit to a letter**, out loud or in writing, before moving on.
- ◆ **Not 'probably B'**. Commit, because an uncommitted guess cannot be wrong and therefore teaches nothing.
- ◆ **Write down why**, in one sentence, because the reasoning is what you are actually testing.
- ◆ **Then check all twelve at the end**, not one at a time, so that the earlier ones do not calibrate the later ones.

THE TAKEAWAY

Checking as you go turns twelve independent tests into one test and eleven adjustments. Do all twelve, then open the file.

The method, as a reminder

QUESTION	WHAT IT RULES OUT
1. Too many rows, too few, or wrong values?	Two thirds of everything, from one row count.
2. Is the discrepancy an exact multiple?	Exact integer means per-level counting. Untidy means data.
3. Does it reconcile against itself?	If yes, the arithmetic is fine and the population is wrong.
4. What does the defect correlate with?	Ties, nulls, group edges, the start of a period.

THE TAKEAWAY

Module twenty-three's four questions. Every one of the twelve that follow is crackable with them, and most of them turn on question four.

What these twelve are drawn from

- ◆ **All twenty-two teaching modules**, in no order, with nothing indicating which one any of them came from.
- ◆ **Several are from Part 1**, which is now a very long way behind you. That is deliberate.
- ◆ **And every one is present in the capstone database**, so you can reproduce it rather than take my word.
- ◆ **Which is the better check.** The answer file says what; the database lets you see it.

THE TAKEAWAY

If you disagree with an answer, the capstone database settles it. That is a better arrangement than my authority, and it is why the data ships with the course.

A nightly extract of the 500 highest-value transactions is compared against the previous night's, on a day when no new data was loaded. 38 transactions differ between the two runs. The extract orders by `net_amount` descending and takes the first 500. Many transactions share the same `net_amount`.

What is the cause?

- ☐ A The source table was updated between the two runs, despite the load not running.
- ☐ B The engine returned rows in a different physical order, which changed the aggregate.
- ☐ C The ordering does not uniquely identify a row, so the set of ties at the cut-off is arbitrary.
- ☐ D A parallel plan split the work differently, producing a different set of 500.

◆ PAUSE AND DECIDE

One warning before you start

- ◆ **Not every one of the twelve has a defect.** At least one is a correct result that somebody has queried.
- ◆ **Which is realistic**, because a query about a wrong number is far more common than a wrong number.
- ◆ **And it changes how you read them**, since assuming a bug exists is how you find one that does not.
- ◆ **'The query is right and the expectation was wrong' is a legitimate answer**, and it is one of the four options somewhere.

THE TAKEAWAY

Knowing that one of them is fine is a real hint and it makes the exercise better, because it stops the search being a hunt for a culprit.

Commit to a letter **before** you want to. The discomfort is the exercise.

A 'days since this member last shopped' column is populated on every sales line, including the 38% of lines that are guest transactions with no member recorded.

The guest figures look plausible, averaging around four days. The window is PARTITION BY member_durable_key ORDER BY sold_ts.

What is the cause?

- ☐ A All the null member keys collect into one partition, so each guest is being compared with an unrelated guest.
- ☐ B LAG returns the previous row of the table when the partition key is null, ignoring the partition entirely.
- ☐ C The guest lines were joined to a default member record, so they all share one member key.
- ☐ D The column is being populated by a later COALESCE that fills nulls with the table-wide average gap.

◆ PAUSE AND DECIDE

A basket-value distribution reports $p_{25} = 48.10$, $p_{50} = 31.75$ and $p_{95} = 12.40$ for every region. The minimum and maximum look correct.

The query uses `PERCENTILE_CONT` with a `WITHIN GROUP` clause. No error is raised and the figures are stable across runs.

What is the cause?

- ☐ A The percentile arguments are in the wrong order, so 0.95 is being passed where 0.25 was meant.
- ☐ B Null basket values are being included, dragging the higher percentiles down.
- ☐ C `PERCENTILE_CONT` is interpolating across region boundaries because the partition is missing.
- ☐ D The `WITHIN GROUP` ordering is descending, so every percentile is measured from the wrong end.

◆ PAUSE AND DECIDE

A query wraps a seven-million-row fact table in a CTE that selects everything, then filters the CTE for a single store. It reads the whole table every time.

Rewriting it so the filter sits directly on the fact table makes the same query effectively instant. The results are identical.

What is the cause?

- ☐ A The CTE version builds a hash table of all seven million rows before filtering it.
- ☐ B The CTE is a materialisation fence, so the filter never reaches the scan and no blocks can be skipped.
- ☐ C The rewritten version can use an index on `store_key`, which the CTE version cannot.
- ☐ D The CTE is being evaluated once per output row, which is why it reads the table repeatedly.

◆ PAUSE AND DECIDE

An order-line table built by unnesting a JSON items array contains 4,180 fewer distinct orders than the order fact. Every line present is correct, and the line totals reconcile to the order values for the orders that are there.

The missing orders all have a value and a customer. No error is raised.

What is the cause?

- ☐ A Those orders have an empty or null items array, and the unnest deleted the parent row.
- ☐ B The JSON on those rows failed to parse, so the array came back as an error value and was skipped.
- ☐ C The items arrays on those orders contain nulls, which the unnest filtered out.
- ☐ D A downstream deduplication removed them because their line counts were zero.

◆ PAUSE AND DECIDE

A report listing the stores that make up the top 80% of sales contains 31% of stores. Commercial expected roughly 20%, and have asked for the query to be corrected.

The running share is computed with ROWS UNBOUNDED PRECEDING, ordered by sales descending, and the retained set's share of the total is verified at 80.4%.

What is the cause?

- ☐ A The running share is computed over the wrong partition, so each region's share is being used.
- ☐ B The frame is RANGE rather than ROWS, so stores with equal sales share a running total.
- ☐ C Keeping rows at or below 0.80 stops before reaching 80%; the last store must be included.
- ☐ D Sales are not concentrated enough for a Pareto assumption, so 31% is simply the correct answer.

◆ PAUSE AND DECIDE

A market-share column reads exactly 100.00% on every row of a store-level report. The sales figures on each row are correct, and the report covers all seven regions.

The column is a store's sales divided by a windowed SUM of sales. No filter is applied anywhere in the query.

What is the cause?

- ☐ A The division is integer division, so every result rounds to 1.
- ☐ B The window's PARTITION BY includes store_key, so each store's denominator is its own sales.
- ☐ C The window has no frame, so the denominator covers the current row only.
- ☐ D The SUM is computed after the GROUP BY collapsed each store to one row, so there is nothing left to divide by.

◆ PAUSE AND DECIDE

A recursive walk over an employee reporting line is expected to return about 88,000 rows. It returns 1,760,000 and stops. The engine reports no error.

The walk goes upward, from each employee to their manager. A depth limit of 20 is configured.

What is the cause?

- ☐ A The anchor term is unfiltered, so the walk starts from every employee rather than from the roots.
- ☐ B The recursive term joins on the wrong column, producing a cross join at each level.
- ☐ C A cycle in the reporting line is being walked repeatedly until the depth limit stops it.
- ☐ D Employees with several managers are producing multiple paths, which is correct behaviour.

◆ PAUSE AND DECIDE

A trading-spell report returns 519 spells where operations expect 475. Every extra spell is adjacent to another spell for the same store, with no missing day between them.

The store-day table was rebuilt last week from a new upstream feed. The query uses the row-number difference technique.

What is the cause?

- ☐ A The rebuilt table has duplicate store-days, so ROW_NUMBER advances twice on one date and the anchor shifts.
- ☐ B The refurbishment table is missing 44 closures, so the expected figure of 475 is itself wrong.
- ☐ C The window's ORDER BY has no tiebreaker, so the numbering is non-deterministic.
- ☐ D The new feed includes non-trading stores, which are producing spells of their own.

◆ PAUSE AND DECIDE

A weekly pack pivots net sales by store format. It has reconciled to the penny against itself every week for two years. In June, finance notice the pack's grand total is about 3% below the group figure.

Kestrel opened 40 franchise sites in March. The pack has not been changed and produces no errors.

What is the cause?

- ☐ A The franchise sites have null sales in the source, so they contribute nothing to any aggregate.
- ☐ B The pivot needs a GROUP BY on the new format column before it will include it.
- ☐ C The engine truncated the result at three pivoted columns because of a column limit.
- ☐ D The pivot's IN list names three formats, and a value not in that list is excluded silently.

◆ PAUSE AND DECIDE

March's ledger figures are restated every week through April and into May, each time by a smaller amount. By June they settle. The same pattern repeats every month.

The aggregate is rebuilt incrementally with a three-day lookback on `effective_date`. The ledger records both an `effective_date` and a `posted_ts`.

What is the cause?

- ☐ A The incremental load is not idempotent, so each rerun adds to the previous month's figures.
- ☐ B Postings are back-dated by more than three days, so each run catches a few more of them.
- ☐ C The ledger is being loaded twice for some days, and the duplicates decay as they are cleaned up.
- ☐ D The `effective_date` is being set from a timezone-shifted timestamp, so rows drift between months.

◆ PAUSE AND DECIDE

A seven-day moving average of daily sales is identical for every day within each store, and equals that store's overall daily mean. The daily totals underneath it vary normally.

The window reads `PARTITION BY store_key`, with no `ORDER BY` and no frame clause.

What is the cause?

- ☐ A The `ORDER BY` is on a column that is constant within each store, so every row is a peer of every other.
- ☐ B The average is computed outside the window and joined back, losing the per-day variation.
- ☐ C With no `ORDER BY`, the default frame is the whole partition, so every row averages every day.
- ☐ D The frame was written as `ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING`.

◆ PAUSE AND DECIDE

The answers are in **answers_module_24.md**, with the reasoning and the module each came from.

What the answer file gives you

FOR EACH OF THE TWELVE	WHY
The letter	So you can score yourself.
The reasoning, in a paragraph	So you can check your reasoning, not just your letter.
Which module it came from	So you know where to reread, if you want to.
Which of the four questions cracks it	Because that is the transferable part.
A query against the capstone database	So you can watch it happen rather than take my word.

THE TAKEAWAY

Five things per diagnostic. The last one matters most, because a defect you have reproduced is understood and one you have read about is not.

What reproducing one looks like

```
1 -- from the answer file, for one of the twelve
2
3 SELECT k, ts,
4        LAG(ts) OVER (PARTITION BY k ORDER BY ts) AS prev
5 FROM (VALUES (1,1),(1,2),(NULL,3),(NULL,4),(NULL,5)) v(k,ts);
6
7 k      ts  prev
8 1      1   NULL
9 1      2    1
10 NULL  3   NULL
11 NULL  4    3    <- two unrelated rows, compared
```

11 Five rows, and the defect is visible by eye. Every answer in the file comes with something this size.

THE TAKEAWAY

This is the whole of one of the twelve, reproduced in five rows. Reading the explanation takes ten seconds; running this takes thirty and is worth more.

Scoring yourself honestly

- ◆ **Predict your score before opening the file.** The gap between the prediction and the result is the useful number.
- ◆ **Over-confident is the common direction**, and it is the one that matters, because it is how a wrong query gets shipped.
- ◆ **Score the reasoning separately from the letter**, and count a right letter with wrong reasoning as wrong.
- ◆ **Eight out of twelve with sound reasoning beats eleven with three lucky guesses**, and only one of those transfers.

THE TAKEAWAY

Calibration matters more than the score. Knowing you are right about nine of twelve is more useful than being right about ten and not knowing which.

If several of them caught you

- ◆ **Look at which of the four questions would have got you there**, and whether you asked it.
- ◆ **Most people skip question one**, which is the row count, because reading the SQL feels more like working.
- ◆ **And go back to the module each one came from**, which the answer file names, rather than rewatching everything.
- ◆ **Three wrong from one part is a signal**. Three wrong scattered across seven parts is ordinary.

THE TAKEAWAY

A cluster of failures in one part means a technique you never quite got. A spread means you are still learning the method, which is expected at this point.

The real test is **the next wrong number at work**, which will not have four options underneath it.

Working these with somebody else

- ◆ **Both commit privately first**, then compare, because hearing a confident answer first removes the exercise for the other person.
- ◆ **Argue the reasoning rather than the letter**, which is where the disagreements are actually informative.
- ◆ **Two people disagreeing on a diagnostic usually disagree about the grain**, which is worth noticing.
- ◆ **And it is a better review exercise than reading each other's SQL**, because it tests the same judgement faster.

THE TAKEAWAY

Twelve diagnostics is about an hour for two people, and it surfaces differences in how they each think about grain, nulls and ordering.

What Part 8 has been for

- ◆ **Module twenty-three gave you a method:** four questions, none of which requires reading the query.
- ◆ **This module removed the safety net,** which is the only way to find out whether the method is yours.
- ◆ **And the capstone removes the options as well,** because there are no candidate answers in a brief.
- ◆ **Each step takes away a support.** That is the design, and it is why the parts are in this order.

THE TAKEAWAY

Ninety-two scenarios with the technique named, twelve with nothing named, then ten deliverables with no question asked at all.

Before you move on

- ◆ **Finish the twelve.** The capstone assumes the method rather than teaching it.
- ◆ **Open the answer file once, at the end,** and score both the letters and the reasoning.
- ◆ **Reproduce at least the ones you got wrong,** on the capstone database, because that is where a belief becomes a memory.
- ◆ **Then set aside a weekend,** which is roughly what the capstone takes.

THE TAKEAWAY

The capstone is the first thing in this course with no answer behind it at all. Arriving with the method unpractised makes it considerably harder than it needs to be.

What to do with the ones you got wrong

- ◆ **Separate a wrong letter from wrong reasoning.** Getting the right answer for the wrong reason is the one to worry about.
- ◆ **Which is why you wrote the reason down,** because without it you cannot tell those two apart afterwards.
- ◆ **Reproduce it on the capstone database,** which turns a corrected belief into something you have seen.
- ◆ **And note which of the four questions would have got you there,** because that is the transferable part.

THE TAKEAWAY

A wrong letter with sound reasoning is a small gap. A right letter with unsound reasoning will be wrong next time, on a defect that looks slightly different.

And the ones you got right

- ◆ **Check whether you eliminated or recognised.** Recognising the scenario from the course is not the same as diagnosing it.
- ◆ **Several of these are close variants of ones you have seen**, deliberately, and one or two are not.
- ◆ **The variants test recall. The new ones test method**, and only the second transfers to work.
- ◆ **So the useful score is on the unfamiliar ones**, which is a smaller number and a more honest one.

THE TAKEAWAY

If you found the familiar ones easy and the unfamiliar ones hard, that is the expected result and it tells you exactly what to practise.

Every one of the twelve **ran without error** and produced a plausible number.

What is left

- ◆ **Module twenty-five is the capstone.** A hundred and seventy megabytes, ten deliverables, a harness that marks you.
- ◆ **Module twenty-six is short,** and is about what actually holds once the techniques have faded.
- ◆ **The capstone is the part that is not a video,** and it is the only part that cannot be done by watching.
- ◆ **Which is why it comes last,** and why the solution ships with it rather than being withheld.

THE TAKEAWAY

Two modules left, and one of them is a project. The course stops telling you things after the next twenty minutes.

Module twenty-five hands you the database, the brief, and the harness.